

Carleton University
COMP 4905 – Honours Project – Summer 2026

Neural Network-Based Cheating Detection for CoMas

A Hybrid Vision and Text Pipeline for
Triaging Online Examination Screenshots

Sabateesh Sivakumar
101229774

Supervisor: Professor Darryl Hill
School of Computer Science

September 2026

Abstract

CoMas is the proctoring tool used at Carleton University to monitor online examinations. It periodically captures a screenshot of each student’s desktop, and every one of those images is currently reviewed by hand. A single course offering can produce many thousands of screenshots, which makes exhaustive manual review both slow and inconsistent between reviewers.

This project develops a screening layer that ranks screenshots by how likely they are to show prohibited behaviour, so that a proctor reviews the most suspicious images first rather than reviewing everything. The system addresses two concrete detection tasks: identifying whether an AI coding assistant (GitHub Copilot or Cursor) is active inside Visual Studio Code, and identifying whether a student has navigated away from their Brightspace quiz.

The central technical finding is that these two tasks require fundamentally different methods. An assistant chat panel and a Brightspace quiz page are both identifiable from on-screen text, and a deterministic keyword rule over OCR output handles them accurately and interpretably. Inline “ghost text” — the dimmed code an assistant suggests at the cursor — contains no distinctive words at all, and is invisible to text extraction. Measured on real screenshots captured during development, an OCR-only detector recovered just 3 of 14 true positives. Detecting ghost text therefore requires a visual model, and a ResNet50 convolutional neural network was fine-tuned for this purpose using transfer learning from ImageNet.

Because access to real CoMas data could not be obtained within the project timeline, training data was generated synthetically. Two generators render labelled screenshots programmatically across multiple editor themes, window resolutions and layouts. A deliberate design decision, and one that materially affected the outcome, was to construct *hard negatives*: half of the negative examples show a non-assistant panel docked in the same screen region as a chat panel, preventing the network from learning the trivial rule “panel on the right means cheating.”

On a held-out synthetic set generated with an unseen random seed, the final model detects assistant chat panels with perfect recall and zero false positives on browser pages and non-chat side panels. Ghost-text detection is substantially harder: an ablation isolating this discrimination measured an AUROC of 0.759 at 512-pixel input resolution, improving to 0.856 at 768 pixels. The mechanism is quantified in the report — a ghost-text line survives downscaling as roughly four pixels of image height, whereas a chat panel occupies a third of the screen. At the shipped operating point the system recovers 95.0% of ghost-text

screenshots.

The deliverable is a desktop application that ingests a folder of screenshots, scores them, and presents a ranked review queue with triage controls and an evidence export suitable for attaching to an academic integrity case.

Acknowledgements

I would like to thank my supervisor, Professor Darryl Hill, for his guidance throughout this project, and in particular for the latitude to narrow the project’s scope once it became clear that a general-purpose “suspicious screenshot” classifier was too vague a target to evaluate rigorously. The decision to focus on two concrete, well-defined detection tasks is the single change that most improved the quality of this work.

I would also like to acknowledge the School of Computer Science for the opportunity to pursue an independent project of this scope, and to note that the experience of running into — and having to diagnose — a long sequence of practical engineering problems taught me at least as much as the machine learning content did.

Contents

Abstract	1
Acknowledgements	3
1 Introduction	6
1.1 Chapter Overview	6
1.2 Project Motivation	6
1.3 Project Objectives	7
1.4 Scope Refinement	8
1.5 Contributions	9
1.6 Report Structure	9
2 Background and Related Work	10
2.1 Chapter Overview	10
2.2 Automated Proctoring	10
2.3 AI Coding Assistants On Screen	10
2.4 Transfer Learning for Image Classification	11
2.5 Optical Character Recognition	12
3 System Design	13
3.1 Chapter Overview	13
3.2 Use Cases	13
3.3 Processing Pipeline	14
3.4 Technologies	14
3.5 Module Structure	15

4	Detection Methods	17
4.1	Chapter Overview	17
4.2	VS Code Detector: OCR Keyword Signal	17
4.3	VS Code Detector: CNN Signal	18
4.4	Signal Fusion	19
4.5	Confidence-Based OCR Skipping	19
4.6	Brightspace Detector	20
4.7	Combined Mode	21
5	Training Data Generation	23
5.1	Chapter Overview	23
5.2	The Data Access Problem	23
5.3	Synthetic VS Code Screenshot Generator	24
5.4	Hard Negatives	24
5.5	Preventing Text Leakage	25
5.6	Domain-Gap Augmentation	26
6	Implementation and Performance	27
6.1	Chapter Overview	27
6.2	OCR Caching	27
6.3	Parallel OCR and an Unexpected Result	27
6.4	Region-of-Interest OCR	28
6.5	Batched CNN Inference	29
6.6	Session-Aware Splitting	29
6.7	Testing	29
7	The Reviewer Interface	31
7.1	Chapter Overview	31
7.2	Design Goals	31
7.3	Workflow	31
7.4	Review Features	32
8	Evaluation and Results	33

8.1	Chapter Overview	33
8.2	Experimental Setup	33
8.3	Metrics	33
8.4	OCR Baseline on Real Screenshots	34
8.5	CNN Results	34
8.6	Per-Variant Analysis	35
8.7	Input Resolution Ablation	35
8.8	Operating Point Selection	36
8.9	Threats to Validity	37
9	Development Experience	38
9.1	Chapter Overview	38
9.2	What Worked	38
9.3	Problems Encountered	39
9.3.1	A blank application window	39
9.3.2	A repository that could not be pushed	39
9.3.3	A detector that detected itself	39
9.3.4	Correct rules that were globally wrong	40
9.3.5	Evaluating on training data	40
9.3.6	Data access	40
9.4	What I Would Do Differently	41
10	Conclusion	42
10.1	Summary	42
10.2	Limitations	42
10.3	Future Work	43
	References	44
	A Repository Structure	45
	B Configuration Reference	46
	C Instructions to Run and Test	47

C.1	Environment	47
C.2	Running the Application	47
C.3	Regenerating Data and Retraining	47
C.4	Running the Tests	48
C.5	Command-Line Detection	48

List of Tables

3.1	Technologies used and their roles.	14
3.2	Modules of the <code>comas</code> package.	15
4.1	A representative subset of the strong keyword set.	18
5.1	Composition of the VS Code training set (<code>data_vscode</code>).	25
6.1	OCR parallelisation, 20 synthetic images, 4 cores, 3 workers.	28
8.1	Training runs. Run 1 predates the addition of browser negatives.	34
8.2	Per-variant performance, 768-pixel model, held-out set, threshold 0.50. . . .	35
8.3	Resolution ablation on the held-out set. Only input resolution differs. . . .	36
8.4	Threshold sweep, 768-pixel model, held-out set. Panel recall is 1.000 and hard-negative and browser FPR are 0.000 at all three thresholds.	36

Chapter 1

Introduction

1.1 Chapter Overview

This chapter introduces the problem the project addresses and explains why I thought it was worth solving. Section 1.2 describes how the project began and the review burden that motivates it. Section 1.3 states the project objectives. Section 1.4 documents how the scope changed from my original proposal and why, since that change affects how the rest of the report should be read. Section 1.5 summarises the contributions, and Section 1.6 outlines the remainder of the document.

1.2 Project Motivation

This project started as a suggestion from Professor Hill rather than as an idea of my own, and my first reaction to it was shaped less by the machine learning involved than by having worked as a teaching assistant for three terms.

I should be precise about my own experience here, because it is the honest version and it matters for how I approached the design. I have not personally had to review student screenshots as part of any of my TA duties. What I do have is three terms of familiarity with the shape of TA work — the marking queues, the repetitive passes over hundreds of near-identical submissions, and a fairly good sense of which parts of the job people dread. When the idea was first described to me, my immediate thought was not “this is an interesting classification problem” but “this sounds miserable, and it does not need to be.” Sitting down to click through several thousand screenshots, the overwhelming majority of which show a student quietly working, is exactly the kind of task that burns people out: high volume, low information density, and almost entirely negative results. If a tool could remove the images that are plainly not worth anyone’s time and leave only those that genuinely need human judgement, that struck me as a real improvement in the working lives of colleagues who do carry this responsibility. That framing — as something built for the person doing the reviewing, not simply as a classifier — stayed with me for the rest of the project and is

visible in a number of design decisions later in this report.

The underlying numbers explain why the problem exists at all. CoMas captures desktop screenshots at intervals while a student completes an online examination, so that a proctor can afterwards inspect what was on screen and determine whether anything prohibited was visible. In practice the volume defeats the purpose. A single examination with a few hundred students, each screenshotted every minute or two across a two-hour sitting, produces tens of thousands of images. Reviewing all of them carefully is not feasible, so review becomes shallow, sampled, or both.

Shallow review fails in two directions at once. Genuine misconduct is missed because the one screenshot that showed it was never looked at closely. At the same time, enforcement becomes inconsistent: whether a given student is caught depends on which reviewer happened to examine their images and how much time that reviewer had that week. The inconsistency is arguably the more serious of the two problems, because academic integrity processes have to be defensible, and a process whose outcome depends on reviewer fatigue is difficult to defend.

The insight the whole project rests on is that the reviewer’s task is not really classification — it is *prioritisation*. A TA does not need software that decides whether a student cheated, and I would be uncomfortable building software that claimed to. What they need is the twenty screenshots most worth looking at placed in front of them first, so their limited attention goes where it has the best chance of mattering. The remaining thousands are not deleted or judged; they are simply moved out of the way until someone has reason to look.

This reframing has a technical consequence that turns out to matter a great deal later. A triage system can tolerate a much higher false positive rate than an automated decision system, because a false positive costs a reviewer a few seconds and a single keystroke to dismiss, whereas a false negative defeats the entire purpose of running the system. That asymmetry is not a detail — it directly determines the operating point I eventually shipped, and it is revisited quantitatively in Chapter 8.

Finally, the specific behaviours I targeted reflect what cheating in a programming examination actually looks like now. AI coding assistants such as GitHub Copilot and Cursor are installed by default in many students’ development environments, and they will readily produce a complete solution to a typical undergraduate examination question. A student no longer needs to visit a prohibited website; the prohibited resource is embedded in the editor they are legitimately entitled to use. That makes this a meaningfully different problem from the classic proctoring task of spotting an open browser tab, and it is the problem I found most worth working on.

1.3 Project Objectives

I set out to accomplish the following:

1. Acquire and curate a labelled dataset of examination screenshots, supplemented by

synthetically generated data where real samples prove insufficient.

2. Design and train a binary image classifier distinguishing legitimate from suspicious screenshots, using a convolutional neural network with transfer learning from a pre-trained backbone.
3. Evaluate the classifier using precision, recall, F1-score and AUROC, with particular attention to minimising false negatives so that genuine misconduct is not missed.
4. Deliver a lightweight inference pipeline that processes a batch of screenshots and produces a ranked list of flagged images for manual review.

Chapter 10 assesses the finished system against each of these objectives, including the one I did not fully meet.

1.4 Scope Refinement

My original proposal described a general binary classifier separating “legitimate” from “suspicious” screenshots, with suspicious behaviour characterised loosely as open browser tabs, messaging applications, or unauthorised reference material. Early in the implementation phase this proved to be an unworkable specification, for a reason worth stating plainly: *“suspicious” is not a visual category*. A screenshot showing a browser is suspicious during a closed-book quiz and entirely unremarkable during an open-book take-home. Without a fixed examination context, no consistent labelling function exists, and without a consistent labelling function there is nothing coherent to train against and no defensible way to measure accuracy.

I resisted narrowing the scope for longer than I should have, because the smaller version felt like a retreat from what I had promised. It was not. The general version was not more ambitious, only less well defined, and I could not have produced a single trustworthy number from it.

The project was therefore narrowed to two concrete tasks, each with an unambiguous ground truth:

1. **AI assistant detection.** Is GitHub Copilot or Cursor visibly active in a Visual Studio Code screenshot, either as a chat panel or as an inline ghost-text suggestion?
2. **Brightspace tab-leave detection.** Has the student navigated away from the Brightspace quiz page they are supposed to be on?

Both questions can be answered by looking at a single screenshot, both admit a labelling rule that two different people would apply identically, and both correspond to behaviours that examiners actually care about. The narrowing traded breadth for the ability to make measurable claims, which for a project of this length was the correct trade.

One further deviation from the proposal should be recorded. The proposal specified a web application as the reviewer interface. The delivered interface is a desktop application built with Tkinter. Functionally these are equivalent for the stated purpose — both accept a folder of screenshots and return a ranked review queue — but the desktop implementation avoids requiring a proctor to upload potentially sensitive examination screenshots to a web service, which on reflection is a meaningful privacy advantage for this particular application. An earlier FastAPI-based web version was implemented and then removed once the desktop application superseded it.

1.5 Contributions

The main contributions of this work are:

- A hybrid detection pipeline that combines a fine-tuned CNN with an OCR keyword detector, together with a quantitative demonstration that neither component alone is sufficient.
- Two synthetic screenshot generators producing labelled training data for the two tasks, including a hard-negative construction that prevents a specific and easily-overlooked shortcut solution.
- An empirical characterisation of the ghost-text detection problem, including an input-resolution ablation that isolates the mechanism behind the model’s principal weakness.
- A reviewer-facing desktop application with ranked review, adjustable sensitivity, triage state and evidence export.
- A set of measured engineering results of independent interest, most notably that naïve process-level parallelisation of Tesseract OCR is approximately six times *slower* than sequential execution unless OpenMP threading is explicitly constrained.

1.6 Report Structure

Chapter 2 reviews relevant background. Chapter 3 describes the system architecture and technology choices. Chapter 4 details the detection methods. Chapter 5 covers synthetic data generation. Chapter 6 discusses implementation and performance engineering. Chapter 7 presents the reviewer interface. Chapter 8 reports experimental results. Chapter 9 reflects on the development experience and the problems encountered. Chapter 10 concludes and outlines future work.

Chapter 2

Background and Related Work

2.1 Chapter Overview

This chapter provides the technical background required to follow the rest of the report. Section 2.2 situates the work within automated proctoring. Section 2.3 describes how AI coding assistants appear on screen, which is the observation the entire detection design rests on. Sections 2.4 and 2.5 cover transfer learning and optical character recognition respectively.

2.2 Automated Proctoring

Commercial remote proctoring systems typically combine webcam monitoring, screen capture, and browser lockdown. Lockdown browsers prevent navigation away from an examination page but cannot govern what happens outside the browser, which is precisely the gap that matters for a programming examination conducted in a local development environment. Screen capture is the more general mechanism, and it is what CoMas uses, but it transfers the burden from prevention to review.

Automated analysis of proctoring data raises well-known concerns around false accusation and equity, and these concerns directly shaped the design here. The system is positioned as a triage aid that ranks screenshots for human review, not as an automated decision-maker. No screenshot is ever labelled as cheating by the system alone; the output is an ordering and a numeric score, and a human makes every determination. The interface reinforces this by requiring the reviewer to explicitly mark each flagged image as reviewed or dismissed.

2.3 AI Coding Assistants On Screen

The design of the VS Code detector follows from a single observation: an AI coding assistant manifests in two visually and textually distinct ways.

The chat panel is a docked side panel, conventionally on the right, containing a conversation between the user and the assistant along with a message composer at the bottom. It is large — typically 25–35% of screen width — and it contains distinctive interface text. GitHub Copilot’s panel displays strings such as “GitHub Copilot” and “Copilot Chat”; Cursor’s displays “Plan agent”, “No matching sessions” and “Explore and understand your code”. Both its size and its text make it comparatively easy to detect.

Inline ghost text is the assistant’s autocomplete suggestion, rendered directly in the editor at the cursor position in a dimmed foreground colour. The student presses Tab to accept it. This is by far the more common mode of use during actual coding, and it is much harder to detect. It occupies a single line of the editor, perhaps 15 pixels tall in a screenshot 1000 pixels high. Critically, *it contains no distinctive text*: the suggestion is ordinary source code, indistinguishable in content from code the student wrote. The words “Copilot” or “Cursor” appear nowhere. The only signal is visual — a run of text rendered in a grey that differs from the surrounding syntax highlighting.

This asymmetry drives every subsequent design decision. Text-based detection can find the panel and can never find ghost text. A visual model is not merely preferable for ghost text; it is the only option.

A complication worth noting is that dimmed grey text is not unique to assistants. Source code comments are also rendered in a muted colour in most editor themes. A model that learns “grey text implies assistant” would flag every well-commented file. This informed the construction of the training corpus described in Chapter 5.

2.4 Transfer Learning for Image Classification

Training a deep convolutional network from random initialisation requires datasets far larger than this project could assemble. Transfer learning avoids this by starting from a network pre-trained on a large general-purpose corpus — here ImageNet — and fine-tuning it on the target task. The early convolutional layers of such a network encode generic visual primitives (edges, textures, colour gradients) that transfer readily to new domains, so only the later layers and classification head need to adapt substantially.

This project uses ResNet50, a 50-layer residual network. Residual connections allow gradients to propagate through deep stacks without vanishing, which makes networks of this depth trainable in practice. The final fully-connected classification layer is replaced with a single-output head producing one logit, which is passed through a sigmoid to yield a probability of the positive class.

Screenshots differ from ImageNet photographs in ways worth acknowledging: they are synthetic renderings with hard edges, flat colour regions, and very fine-grained text detail rather than natural textures. That ImageNet features transfer usefully at all is not a given, though the empirical results in Chapter 8 indicate they do. The resolution ablation in Section 8.7 suggests the fine-detail requirement is precisely where the approach strains.

2.5 Optical Character Recognition

OCR converts pixels containing rendered text into machine-readable strings. This project uses Tesseract, a mature open-source engine, via the `pytesseract` Python binding. Screenshots are an unusually favourable input for OCR: the text is crisply rendered at high contrast rather than photographed, so recognition accuracy is high with no preprocessing.

The relevant weakness of keyword-based detection is not accuracy but *brittleness over time*. The keywords are interface strings chosen by a vendor, and vendors rename things. A Copilot update that changes “Tab to accept” to different wording silently disables that detection rule with no error and no warning. This is a genuine maintenance burden and is discussed further in Section 10.2. It is also the main argument for retaining the CNN as the primary signal: the two methods fail under different and largely uncorrelated conditions.

Chapter 3

System Design

3.1 Chapter Overview

This chapter describes the architecture of the delivered system. Section 3.2 states the use cases. Section 3.3 describes the processing pipeline. Section 3.4 lists the technologies used and justifies the significant choices. Section 3.5 documents the module structure of the codebase.

3.2 Use Cases

The system supports a single primary actor, the proctor or teaching assistant reviewing examination screenshots.

Select a detection mode. The reviewer chooses among three modes: *VSCODE Cheating* for programming examinations conducted in an editor, *Brightspace Cheating* for quizzes taken in the browser, and *VSCODE + Brightspace Cheating* for examinations involving both. The mode determines which detectors run and, as described in Section 4.7, how their outputs are interpreted.

Submit a batch of screenshots. The reviewer selects a set of image files through a native file dialog. Analysis runs on a background thread with progress reporting, so the interface remains responsive.

Review a ranked queue. Flagged screenshots are presented in descending order of suspicion score. The reviewer pages through them with buttons or arrow keys, each shown with its score and a plain-language evidence line.

Adjust sensitivity. A threshold slider re-filters results live. The reviewer can loosen the threshold to see marginal cases or tighten it if the queue is too noisy, without re-running the analysis.

Record triage decisions. Each screenshot can be marked reviewed or dismissed, so a

reviewer working through a large queue can track their position.

Export evidence. The reviewer exports a timestamped report folder containing copies of the flagged images, a CSV of all scores, and a summary suitable for attaching to an academic integrity case.

3.3 Processing Pipeline

Analysis proceeds in four stages.

Stage 1: CNN scoring. If a trained checkpoint is available and the mode includes VS Code detection, every screenshot is scored by the CNN. Images are processed in batches of 16 rather than individually, which is discussed in Section 6.5.

Stage 2: Selective OCR. Screenshots the model is already confident about (score ≥ 0.90) skip OCR entirely, since the outcome cannot change. The remainder are OCR'd in parallel across CPU cores. This ordering is deliberate: OCR is by far the most expensive stage, so it runs only where it can still affect the result. Section 4.5 explains why this optimisation is applied only to high-confidence detections and never to confident *negatives*.

Stage 3: Detection. Each screenshot is scored by the enabled detectors, reading from the warmed caches. The final score is the maximum across detectors, so any single strong signal is sufficient to flag an image.

Stage 4: Ranking and presentation. Results are sorted by descending score and passed to the interface, which filters at the active threshold.

3.4 Technologies

Two choices merit justification.

ResNet50 over a Vision Transformer. The proposal raised the possibility of a vision transformer. ResNet50 was chosen because transformers are substantially more data-hungry, and with a synthetic dataset of 800 images the risk of a transformer memorising generator artefacts rather than learning the visual signal was high. ResNet50's stronger inductive bias toward local spatial features is also a better match for a task where the signal is a localised region of the screen.

Tkinter over a web framework. Tkinter ships with Python, requires no server, and keeps examination screenshots on the reviewer's machine. Given that the input is sensitive student data, avoiding a network upload step is a genuine advantage rather than merely a convenience. The cost is a less sophisticated interface toolkit, which required some care to produce an acceptable result and caused one significant portability problem documented in Section 9.3.1.

Table 3.1: Technologies used and their roles.

Technology	Version	Role
Python	3.11	Implementation language
PyTorch	2.x	Model definition, training, inference
torchvision	0.x	ResNet50 backbone and ImageNet weights
Tesseract	4.1	Optical character recognition
pytesseract	0.3	Python binding for Tesseract
Pillow (PIL)	10.x	Image loading, rendering, augmentation
Tkinter	Tk 8.6	Desktop user interface
scikit-learn	1.x	Evaluation metrics
PyYAML	6.x	Configuration file parsing
pytest	7.x	Automated testing

3.5 Module Structure

The implementation lives in a single Python package, `comas/`, comprising roughly 15 modules.

The separation between detection logic and interface is strict: no module in the detection path imports from `gui.py`, and every detector is usable from the command line independently of the interface. This made the detection logic testable without instantiating a window, which matters because a graphical toolkit cannot be exercised in a headless environment.

Table 3.2: Modules of the `comas` package.

Module	Responsibility
<code>copilot.py</code>	VS Code detector: OCR keywords, CNN scorer, signal fusion
<code>brightspace.py</code>	Brightspace detector: quiz-marker rule, IDE recognition
<code>ocr.py</code>	Tesseract wrapper, content-addressed cache, parallel batching
<code>model.py</code>	ResNet50 and fusion model definitions
<code>data.py</code>	Dataset, transforms, augmentation, session-aware splitting
<code>train.py</code>	Training loop, checkpoint selection
<code>evaluate.py</code>	Metric computation
<code>infer.py</code>	Command-line batch inference
<code>config.py</code>	Typed configuration loaded from YAML
<code>gui.py</code>	Desktop reviewer interface
<code>synthetic.py</code>	Synthetic VS Code screenshot generator
<code>synthetic_brightspace.py</code>	Synthetic browser and quiz generator
<code>variant_report.py</code>	Per-variant evaluation tool
<code>realpairs.py</code>	Utilities for organising real screenshot pairs

Chapter 4

Detection Methods

4.1 Chapter Overview

This chapter details how each detector works. Section 4.2 describes the OCR keyword detector and Section 4.3 the CNN. Section 4.4 explains how their outputs combine and Section 4.5 the confidence-based optimisation applied to that combination. Section 4.6 covers the Brightspace detector, whose design changed substantially during development, and Section 4.7 the interaction between detectors in combined mode.

4.2 VS Code Detector: OCR Keyword Signal

The keyword detector extracts text from a screenshot and searches for strings characteristic of an assistant interface. Keywords are partitioned into strong and weak sets. A strong keyword is one that essentially cannot appear except in an assistant panel; a match scores 0.95. Weak keywords are suggestive but individually inconclusive; a single match scores 0.45 and two or more score 0.75.

Each keyword also maps to the tool it indicates, so the detector reports *which* assistant was seen. This distinction proved practically relevant: the real screenshots captured during development showed Cursor rather than Copilot, and a detector built only around Copilot's interface strings would have missed them entirely.

An instructive bug shaped this keyword set. The bare word `copilot` was initially included as a keyword. During testing on real screenshots it produced false positives on screenshots that showed no assistant at all. The cause was that this project's own source file is named `copilot.py`, and that filename was visible in the VS Code file explorer sidebar. OCR read it, the substring matched, and the screenshot was flagged. The fix was to remove the bare word and require multi-word phrases specific to assistant interfaces.

The episode is worth recording because it illustrates a general hazard of keyword detection: the detector has no notion of *where* on screen a string appeared or what interface element

Table 4.1: A representative subset of the strong keyword set.

Keyword	Tool indicated
github copilot	Copilot
copilot chat	Copilot
tab to accept	Copilot
accept suggestion	Copilot
plan agent	Cursor
no matching sessions	Cursor
explore and understand your code	Cursor
before implementing changes	Cursor

it belonged to. It matches text anywhere in a flattened transcript of the entire screen. A file listing, a browser bookmark, or a lecture slide mentioning Copilot would all trigger it equally. Multi-word interface phrases mitigate this but do not eliminate it, and a regression test now pins the behaviour:

```

1 def test_filename_copilot_does_not_trigger():
2     text = "EXPLORER __init__.py copilot.py data.py config.yaml"
3     strong, weak = find_keywords(text)
4     assert strong == []
5     assert weak == []

```

Listing 4.1: Regression test for the filename false positive.

4.3 VS Code Detector: CNN Signal

The CNN is a ResNet50 initialised from ImageNet weights, with its final classification layer replaced by a dropout layer followed by a single-output linear head. It is trained to distinguish screenshots showing an assistant — either as ghost text or a chat panel — from screenshots that do not.

Training uses binary cross-entropy with logits. Because the dataset contains 200 positives against 600 negatives, the loss is weighted by the ratio of negative to positive examples, computed automatically as 3.06, so the model is not rewarded for simply predicting the majority class. Optimisation uses AdamW with a learning rate of 1×10^{-4} , weight decay of 1×10^{-4} , and a cosine annealing schedule across 10 epochs.

The checkpoint retained is not the final epoch but the epoch with the best validation AUPRC. Area under the precision-recall curve was chosen over accuracy for selection because it is the metric most sensitive to performance on the minority positive class, which is the class the system exists to find. Training runs on Apple Silicon GPU via PyTorch’s Metal Performance Shaders backend, falling back to CUDA or CPU as available.

```

1 class CNNOnlyModel(nn.Module):
2     def __init__(self, backbone="resnet50", dropout=0.3, pretrained=True
3     ):
4         super().__init__()
5         self.backbone, feat_dim = _build_cnn_backbone(backbone,
6         pretrained)
7         self.head = nn.Sequential(
8             nn.Dropout(dropout),
9             nn.Linear(feat_dim, 1),
10        )
11
12    def forward(self, x, text_emb=None):
13        return self.head(self.backbone(x)).squeeze(-1)

```

Listing 4.2: Model construction. `use_ocr` selects between the CNN-only and fusion architectures.

A second architecture, `FusionModel`, is implemented and available. It concatenates the CNN feature vector with a sentence-transformer embedding of the screenshot’s OCR text before the classification head, allowing the network to learn jointly from appearance and text. It was not used for the final results: with a synthetic training set whose panel text is deliberately generic (Section 5.5), the text branch carries little information, and the added dependency and inference cost were not justified. It remains as infrastructure for future work with real data, where text content would be far richer.

4.4 Signal Fusion

The two signals combine by taking the maximum. If either the CNN or the keyword detector produces a high score, the screenshot is flagged. This is deliberately permissive, and follows from the asymmetry discussed in Section 1.2: the cost of a false positive is a few seconds of reviewer attention, while the cost of a false negative is undetected misconduct.

Maximum was chosen over a weighted average because the two detectors have disjoint competencies rather than overlapping opinions. The CNN can see ghost text and the keyword detector cannot; the keyword detector can identify an assistant the CNN was never trained on and name it explicitly. Averaging would let one detector’s justified ignorance dilute the other’s justified confidence. Averaging a CNN score of 0.95 with a keyword score of 0.0 yields 0.475 — a screenshot with visible ghost text would fall below any reasonable threshold purely because the text detector, which structurally cannot see ghost text, did not vote for it.

4.5 Confidence-Based OCR Skipping

OCR dominates runtime, costing roughly 5.8 seconds per screenshot at native resolution against approximately 0.3 seconds for CNN inference. Since the CNN runs first, its output can be used to skip OCR where OCR cannot change the outcome.

The rule implemented is deliberately *one-sided*. If the CNN scores a screenshot at 0.90 or above, OCR is skipped: the image will be flagged regardless, so the additional evidence would not alter the decision. If the CNN scores *low*, OCR still runs.

The asymmetry is essential and was the subject of careful reasoning during implementation. It is tempting to skip OCR on confident negatives as well — it would roughly double the saving, since most screenshots in a real stream are negatives. But the CNN was trained to recognise ghost text and chat panels. A screenshot showing an assistant interface the model has never seen, or one whose theme falls outside the training distribution, may score low while containing an unambiguous “GitHub Copilot” string in plain text. Allowing a low CNN score to suppress OCR would convert a certain detection into a silent miss. The behaviour is pinned by test:

```
1 def test_low_model_score_never_suppresses_ocr():
2     # the model only learned ghost text, so it must not veto an open
   chat panel
3     cache = _CountingCache("GitHub Copilot")
4     det = CopilotDetector(ocr_cache=cache, ml_scorer=lambda p: 0.02)
5     ev = det.detect(Path("x.png"))
6     assert cache.calls == 1
7     assert ev.score == 0.95
```

Listing 4.3: The model must never be able to veto a keyword detection.

4.6 Brightspace Detector

The Brightspace detector uses no machine learning. It is a deterministic rule over OCR output, and this is a considered choice rather than an omission: a quiz page is identifiable by its text, so a rule is accurate, interpretable, requires no training data, and can be corrected immediately if Brightspace changes its interface. The CNN is applied where learning is necessary; where a deterministic rule suffices, a rule is the better engineering choice.

The detector’s design changed substantially during development, and the change is more interesting than the final rule.

Initial design (blocklist). The first implementation searched for off-task site markers — `chatgpt`, `stackoverflow`, `google.com/search`, `chegg`, `quizlet` and similar — and flagged screenshots containing them. Quiz markers such as “Submit Quiz”, “(1 point)” and “elapsed” were used to identify legitimate quiz pages, with a `split_view` verdict when both appeared.

This had two failure modes. First, it could only detect sites on the list; a student consulting a site the author had not thought of was invisible. Second, and more seriously, it produced false positives on legitimate quiz pages, because quiz questions themselves mention websites. A question asking which source is acceptable to cite might contain the word “Wikipedia”; a question about research methods might mention Google. The student is doing exactly what they should be, and the detector flags them.

Final design (allowlist). The rule was inverted. Rather than enumerating prohibited

states, the detector recognises the single legitimate one:

$$\text{verdict} = \begin{cases} \text{on_quiz} (0.05) & \text{if quiz markers are present} \\ \text{left_quiz} (0.95) & \text{otherwise} \end{cases}$$

During a Brightspace examination the quiz should be on screen. Any capture without it represents a departure, whether to ChatGPT, a messaging application, a photo library, or anything else. This catches sites never enumerated, and it cannot be fooled by quiz text that happens to mention a website, because quiz markers are decisive.

The off-task keyword list was retained, but demoted from determining the verdict to annotating it. When a tab-leave is detected and a recognised site appears in the text, the evidence line reads `left_quiz (stackoverflow)`; otherwise it reads `left_quiz (unrecognized page)`. The reviewer gets the useful detail without the detector depending on it.

The trade-off is that a student displaying the quiz and a prohibited site side-by-side in one screenshot is classified as `on_quiz`. This is the price of not flagging quizzes that mention websites, and it was judged the correct trade: the split-screen case is still caught by any capture in which the quiz is scrolled away or covered, whereas the false-positive case would affect honest students on every question that references a source.

4.7 Combined Mode

Running both detectors together introduced an interaction bug worth documenting, since it illustrates how a locally correct rule can be globally wrong.

Under the allowlist rule, any screenshot without quiz markers is a tab-leave scoring 0.95. In a combined examination — a programming test with a Brightspace component — a VS Code screenshot contains no quiz markers, so every legitimate coding screenshot was flagged as having left the quiz. The detector was behaving exactly as specified, but the specification implicitly assumed a browser-only examination.

The fix adds context-awareness. In combined mode, a screenshot recognised as a VS Code window contributes nothing from the Brightspace detector; the VS Code detector judges it instead. Recognition uses a small set of IDE interface strings, requiring at least two matches so that a quiz question mentioning a Python file does not accidentally qualify:

```
1 IDE_KEYWORDS = [  
2     "explorer", "debug console", "problems", "output", "terminal",  
3     "spaces: 4", "utf-8", ".py", "outline", "timeline",  
4 ]  
5  
6 def looks_like_ide(text: str) -> bool:  
7     if not text:  
8         return False  
9     low = text.lower()  
10    return sum(1 for k in IDE_KEYWORDS if k in low) >= 2
```

Listing 4.4: IDE recognition used to resolve the cross-detector conflict.

The exemption applies only in combined mode. In Brightspace-only mode an IDE screenshot receives no special treatment, because in a browser-based examination an open code editor is itself worth a reviewer’s attention. Verified against the real VS Code screenshots collected during development, all five were correctly recognised as IDE windows and contributed zero Brightspace score in combined mode.

Chapter 5

Training Data Generation

5.1 Chapter Overview

This chapter describes how training data was produced. Section 5.2 explains why synthetic data was necessary. Section 5.3 describes the VS Code generator and Section 5.4 the hard-negative design, which is the most consequential decision in the chapter. Section 5.5 covers prevention of text leakage and Section 5.6 the domain-gap augmentations.

5.2 The Data Access Problem

The proposal identified obtaining labelled data as the project’s principal risk, and that assessment proved correct. Real CoMas screenshots contain student work, personal information, and in some cases visible identity documents. Access requires institutional data-sharing arrangements and research ethics approval, processes with lead times measured in months and largely outside a student’s control. Within the project timeline this data did not become available.

The proposal anticipated this and specified a fallback: generating synthetic data by staging deliberately non-compliant desktop environments. That fallback became the primary strategy. Three sources of data were ultimately used:

1. **Synthetic screenshots**, rendered programmatically. These form the training and validation sets. Roughly 800 VS Code images and 400 browser images.
2. **Real staged screenshots**, captured manually on the author’s own machine with Cursor active and inactive. Approximately 40 images, used exclusively for qualitative validation — never for training.
3. **Held-out synthetic screenshots**, generated after training with an unseen random seed, used for the quantitative evaluation in Chapter 8.

The limitation this imposes on the results is significant and is stated explicitly in Section 8.9 rather than left for a reader to infer.

5.3 Synthetic VS Code Screenshot Generator

The generator, `synthetic.py`, renders VS Code screenshots directly with Pillow’s drawing primitives rather than automating a real editor. Programmatic rendering gives exact ground truth — the generator knows precisely which pixels are ghost text — and allows systematic variation across conditions that would be tedious to stage manually.

Each generated screenshot varies along several dimensions: four editor themes (Dark+, One Dark, Monokai, Light+), four window resolutions from 1280×800 to 1680×1050 , font size, the presence and width of the file explorer sidebar, the number and arrangement of open editor tabs, cursor position, and scroll offset.

The code displayed in the editor is drawn from the project’s own Python source files and tokenised for syntax highlighting with a simplified lexer covering keywords, strings, numbers, comments and function names. Using real source code rather than generated text keeps the visual statistics realistic — genuine indentation patterns, line-length distributions and identifier shapes.

Comments are deliberately retained in the corpus. Since comments render in a muted colour similar to ghost text, removing them would allow the network to learn the shortcut “any grey text implies assistant”. Their presence forces it to distinguish ghost text from comments on finer cues: position relative to the cursor, the absence of a comment marker, and continuation of a code statement rather than prose.

Positive examples come in three variants: *ghost* (an inline suggestion only), *panel* (a chat panel only), and *both*. The chat panel is rendered as a docked right-hand region with a header, alternating user and assistant message bubbles, occasional embedded code blocks, and a composer box at the bottom.

5.4 Hard Negatives

The most consequential design decision in the dataset concerns what the negative examples look like.

The naïve approach is to pair chat-panel positives with clean-editor negatives. The problem is that this makes the classes separable by a feature that has nothing to do with assistants: *the presence of a docked panel on the right*. VS Code has many such panels — Outline, Extensions, Source Control, Search, split editors. A model trained against clean editors alone learns “panel on the right implies cheating” because that rule achieves perfect training accuracy, and would then flag every student with the Extensions view open.

To prevent this, half of the negative examples render a non-assistant panel in the same screen

region, at the same range of widths, with the same header styling. The model cannot use position or the mere existence of a panel; it must learn the chat panel’s internal structure — stacked message bubbles above a composer box — to separate the classes.

A second category of negative was added later, after the first training run. The initial dataset’s negatives were all VS Code editors, meaning the model had never seen a screenshot that was not an IDE. A real CoMas capture stream is mostly browsers, document editors and desktops. A model whose entire experience is editors has undefined behaviour on a browser window, and could plausibly flag every Brightspace quiz page as an assistant. The 400 synthetic browser screenshots from the Brightspace generator — all assistant-free by construction — were therefore folded in as additional negatives.

Table 5.1: Composition of the VS Code training set (`data_vscode`).

Class	Variant	Count	Content
Positive	ghost	80	Inline suggestion at cursor
Positive	panel	80	Assistant chat panel docked right
Positive	both	40	Ghost text and panel together
Negative	clean	100	Editor, no panel, no ghost text
Negative	hardneg	100	Non-assistant panel docked right
Negative	browser	400	Quiz and non-quiz browser pages
Total		800	

The variant of each image is encoded in its filename, which enables the per-variant analysis in Section 8.6. This was added after an earlier training run produced a single aggregate accuracy figure from which it was impossible to determine whether the model detected panels, ghost text, or only one of the two. Aggregate metrics conceal exactly the failure that matters most here.

5.5 Preventing Text Leakage

A subtle hazard in generating synthetic data for a vision model is that the rendered text may make the task trivially solvable by reading rather than seeing. If the synthetic chat panels displayed “GitHub Copilot” in their headers, a fusion model’s text branch could separate the classes perfectly without the CNN learning anything visual, and reported accuracy would say nothing about ghost-text detection.

Two measures prevent this. Source files whose names or contents contain `copilot`, `cursor` or `ghost` are excluded from the code corpus, and matching lines are filtered out. Chat panel text is deliberately generic: prompts are ordinary programming questions, replies are ordinary explanations, and the composer placeholder reads “Ask anything” rather than any vendor-specific string.

This was verified empirically. Running the OCR keyword detector across a sample of the generated dataset flagged 0 of 15 positives and 0 of 15 negatives, confirming that no keyword signal is available and the CNN must rely on visual structure. A consequence worth stating is that these synthetic images are useless for evaluating the OCR detector — that evaluation requires real screenshots, and is reported separately in Section 8.4.

5.6 Domain-Gap Augmentation

Synthetic renders are pixel-perfect. Real screenshots have been captured from a high-DPI display, possibly rescaled, and possibly compressed. A model trained exclusively on crisp renders may key on that crispness.

Three train-time augmentations narrow this gap: random downscale-then-upscale (simulating resolution loss), random JPEG compression at quality 40–90 (introducing block artefacts), and slight Gaussian blur. Each applies probabilistically, so the model sees both degraded and clean versions across epochs.

These are implemented as plain classes rather than lambdas for a specific practical reason. PyTorch’s `DataLoader` with multiple worker processes must serialise transforms to those workers, and on macOS workers are spawned rather than forked, so transforms must be picklable. A lambda is not picklable and would crash training with a multi-worker configuration. A test asserts picklability so this cannot silently regress.

An honest note on these augmentations: they narrow the domain gap in one direction but also actively suppress the ghost-text signal, since a one-pixel line of dimmed text is exactly what blur and compression destroy first. The resolution ablation in Section 8.7 should be read with this interaction in mind.

Chapter 6

Implementation and Performance

6.1 Chapter Overview

This chapter covers implementation decisions that materially affected correctness or performance. Section 6.2 describes the OCR cache. Section 6.3 reports the parallelisation work, including a result that inverted the expected outcome. Section 6.4 covers region-of-interest OCR, Section 6.5 batched inference, and Section 6.6 session-aware data splitting. Section 6.7 describes the testing strategy.

6.2 OCR Caching

OCR is expensive and its result depends only on image content, making it an ideal caching target. The cache is keyed by SHA-1 hash of the file's bytes rather than by path, so identical images cached under one filename are found under another, and modified files miss correctly.

Two robustness properties were added after considering realistic failure. The cache flushes to disk every 25 entries rather than only at completion: a crash or a closed window at image 190 of 200 would otherwise discard all completed work. Writes go to a temporary file followed by an atomic rename, so an interruption mid-write cannot corrupt the cache and lose every previously cached result.

Region crops are cached under distinct keys from full-frame extractions, so that a partial-image transcript can never satisfy a later full-image lookup.

6.3 Parallel OCR and an Unexpected Result

Tesseract is single-threaded per image and OCR dominates runtime, so distributing images across a process pool appeared to be a straightforward optimisation. Measured on a 4-core machine, the first implementation processed 20 images in 29.6 seconds against 4.6 seconds

sequentially — approximately **six times slower**.

The cause is OpenMP. Tesseract 4 is internally multi-threaded, and each worker process spawned its own thread pool. With several workers each spawning threads on a 4-core machine, the resulting oversubscription caused contention that overwhelmed any benefit from parallelism. Constraining each worker to a single OpenMP thread resolved it. The constraint is applied inside a pool initialiser so that it is automatic rather than dependent on the user's environment:

```
1 def _worker_init():
2     # Tesseract multithreads through OpenMP. Left alone, every worker
      spawns
3     # its own thread pool and they fight over the cores - measurably
      slower
4     # than running sequentially.
5     os.environ["OMP_THREAD_LIMIT"] = "1"
```

Listing 6.1: Without this initialiser, parallelisation is a large performance regression.

Table 6.1: OCR parallelisation, 20 synthetic images, 4 cores, 3 workers.

Configuration	Time (s)	Speedup
Sequential	4.6	1.00×
Parallel, unconstrained OpenMP	29.6	0.16×
Parallel, one thread per worker	2.6	1.79×
Parallel + region-of-interest	2.2	2.13×

On real screenshots at 3600×2338 , six images took 17.6 seconds sequentially and 12.4 seconds in parallel, a more modest $1.42\times$. The smaller gain reflects memory bandwidth limits when several workers each hold a large image. The general lesson — that a plausible optimisation can be a substantial regression, and that measurement rather than reasoning settles the question — recurred often enough in this project to be worth stating once plainly.

A related measurement informed a decision not taken. Downscaling screenshots before OCR reduced per-image time from 5.84 to 3.09 seconds, an attractive $1.89\times$. But on real screenshots the keyword verdict changed on 2 of 6 images, including one true positive dropping from 0.95 to 0.00, because small interface text ceased to be legible. The optimisation was rejected. It would have been easy to adopt on the strength of the timing number alone without checking whether the output was still correct.

6.4 Region-of-Interest OCR

For Brightspace detection, the decisive evidence — tab titles and the URL bar — sits in the top portion of the screen. OCR'ing only the top 25% is correspondingly faster.

Measured directly, cropping alone was not safe: it disagreed with full-frame analysis on 2 of 12 images, in each case downgrading a genuine tab-leave because that particular page had nothing recognisable in its top strip. The failure mode was specific — the crop found nothing *at all* — which made it escapable. The implemented approach is two-stage: OCR the top strip first, and if it yields no quiz markers, re-read the full frame before concluding anything. Verdicts then matched full-frame analysis on 16 of 16 test images while retaining most of the speed benefit.

6.5 Batched CNN Inference

The interface originally scored screenshots one at a time in a Python loop, performing a full forward pass on a batch of one. Since the fixed overhead of a forward pass is substantial relative to the marginal cost of additional images in a batch, images are now stacked into batches of 16.

The scorer remains callable per-image for compatibility with the command-line tools, but the interface uses `score_batch`. An unreadable or corrupt file scores 0.0 and logs a warning rather than raising, so one bad file in a folder of 500 cannot abort an entire review session.

6.6 Session-Aware Splitting

Screenshots from the same examination session are highly correlated: same theme, same window size, same file open, often consecutive frames differing by a few lines of scroll. Splitting such data randomly leaks information between training and test sets, because a test image may be nearly identical to a training image, and reported accuracy becomes optimistic.

Splitting is therefore performed at the session level. Each filename begins with a session identifier, and all images from a session are assigned wholly to one split. The generator distributes images across 20 synthetic sessions for this purpose.

This principle initially had a gap in the evaluation tooling. The per-variant report tool originally scored every image in the dataset, including the 560 the model had trained on, inflating every figure it produced. The tool now reproduces the exact split used during training — same seed, same fractions — and defaults to reporting only held-out test sessions.

6.7 Testing

The project has 42 automated tests covering keyword matching, scoring rules, cache behaviour, augmentation properties, the synthetic generator’s label balance, and interface logic.

Two testing decisions are worth noting. Pure logic was deliberately separated from interface code as static methods, so that scoring and evidence-formatting rules can be tested without instantiating a window — graphical toolkits cannot be exercised in a headless environment.

And every behavioural bug found during development had a regression test added alongside its fix, including the `copilot.py` filename false positive, the one-sided OCR skip rule, and the combined-mode IDE exemption. These tests encode decisions whose rationale is not obvious from the code alone, and which a future maintainer might otherwise reasonably reverse.

Chapter 7

The Reviewer Interface

7.1 Chapter Overview

This chapter describes the delivered application. Section 7.2 states the design goals, Section 7.3 walks through the workflow, and Section 7.4 details the review features.

7.2 Design Goals

The interface was designed around three principles.

The reviewer decides. The system never labels a screenshot as cheating. It produces a score and an ordering, and every determination is made by a human. This is reflected in the vocabulary — screenshots are “flagged” for review, not identified as violations — and in requiring explicit triage decisions.

Sensitivity is the reviewer’s choice. Different courses have different tolerances, and the appropriate trade-off between missed detections and reviewer time is a policy question rather than a technical one. The threshold is exposed as a live control rather than fixed in code.

Institutional visual identity. The application follows Carleton University’s brand: the official wordmark, Carleton red (`#E91C24`), and a flat, clean layout. For a tool intended for administrative use, looking like institutional software rather than a student project is not a trivial concern.

7.3 Workflow

Mode selection. On launch the reviewer chooses one of three detection modes. The upload control remains disabled until a mode is chosen, making the required order unambiguous.

Upload and analysis. A native file dialog accepts multiple images. Analysis runs on a

background thread so the window stays responsive, and the status line reports each stage — model scoring, text extraction with the number of cores in use, and final analysis — with running counts.

Review. Flagged screenshots appear in descending score order. Each is displayed on a dark stage that expands to fill available window space and re-fits when the window is resized. Images are resampled with the Lanczos filter, which preserves the legibility of small interface text — important when the evidence under review may be a single line of dimmed code.

7.4 Review Features

Threshold slider. Adjusting the threshold re-filters results immediately without re-running detection. The implementation stores raw per-detector scores rather than pre-computed verdicts, and derives displayed evidence at render time, so the evidence line stays consistent with the active threshold.

Triage state. Each screenshot can be marked reviewed or dismissed, displayed as a badge and carried into the export. Keyboard shortcuts (**r**, **d**, **c**, arrow keys) allow a reviewer to work through a long queue without the mouse.

Evidence export. Export produces a timestamped folder containing copies of flagged images excluding dismissed ones, a CSV recording every screenshot with per-detector scores, flag state, triage decision and evidence text, and a summary file recording the mode, threshold and counts. Recording the threshold matters for auditability: a reviewer’s decisions are only interpretable alongside the sensitivity at which they were made.

Chapter 8

Evaluation and Results

8.1 Chapter Overview

This chapter reports experimental results. Section 8.2 describes the setup and Section 8.3 the metrics. Section 8.4 establishes the OCR baseline on real screenshots — the measurement that motivates the CNN. Section 8.5 reports overall CNN performance and Section 8.6 the per-variant breakdown that reveals where it actually succeeds. Section 8.7 presents the resolution ablation and Section 8.8 the operating point selection. Section 8.9 states threats to validity.

8.2 Experimental Setup

All experiments ran on an Apple Silicon MacBook Pro using PyTorch’s Metal Performance Shaders backend. Training used 10 epochs at batch size 8 for 768-pixel input, approximately five minutes per epoch.

Three datasets are used. **Training** (`data_vscode`): 800 images, split 560 train / 150 validation / 90 test by session. **Held-out synthetic** (`data_vscode_holdout`): 255 images generated after training with random seed 777, so no image and no session was seen during training — 100 positives (40 ghost, 40 panel, 20 both) and 155 negatives (50 clean, 50 hard-negative, 55 browser). **Real screenshots**: approximately 40 images captured manually with Cursor active or absent, used for the OCR baseline and qualitative validation.

8.3 Metrics

Recall is emphasised throughout, per the objective of minimising false negatives. AUROC and AUPRC are reported as threshold-free summaries, with AUPRC preferred for checkpoint selection because it is more sensitive to minority-class performance.

One additional metric was defined for this project. **Ghost-vs-clean AUROC** is computed using only ghost-text positives and clean-editor negatives, excluding panels and browsers. Overall AUROC is dominated by easy comparisons — a chat panel against a browser page is trivially separable — and conceals performance on the hardest discrimination, which is a dimmed line of code against a plain editor. This metric isolates it.

8.4 OCR Baseline on Real Screenshots

The keyword detector was evaluated on the real staged screenshots. Of 14 real positives, it flagged 3 — a recall of 0.214. On a further 5 real positives held out separately, it flagged 0.

Inspection of the failures confirmed the predicted mechanism. The three detections all had an assistant chat panel or tooltip open, supplying matching interface text. The eleven misses showed inline ghost text with no panel: a dimmed line of suggested code beside a small gutter icon, with no assistant text anywhere on screen. The OCR transcript of these images contains only the student’s source code, which is indistinguishable from any other code.

This measurement is the empirical justification for the CNN. It also indicates that a keyword-only system — which would be far simpler to build and maintain — would miss roughly four out of five real cases.

8.5 CNN Results

Three training runs were conducted.

Table 8.1: Training runs. Run 1 predates the addition of browser negatives.

Run	Res.	Images	Test AUROC	Test AUPRC	Acc@0.5
1	512	400	0.941	0.956	0.900
2	512	800	0.996	0.958	0.967
3	768	800	0.994	0.944	0.978

These aggregate figures are reported for completeness but should be treated with caution, for two reasons. The test split contains only 90 images, of which few are ghost-text positives, so aggregate metrics are dominated by easy negatives. More importantly, aggregate accuracy conceals the distinction that matters — whether the model detects ghost text or only panels. That question requires the per-variant analysis below.

8.6 Per-Variant Analysis

Evaluating the 768-pixel model on the held-out synthetic set, broken down by variant, gives a much clearer picture.

Table 8.2: Per-variant performance, 768-pixel model, held-out set, threshold 0.50.

Class	Variant	Count	Recall / FPR
Positive	panel	40/40	recall 1.000
Positive	both	20/20	recall 1.000
Positive	ghost	28/40	recall 0.700
Negative	hardneg	0/50	FPR 0.000
Negative	browser	0/55	FPR 0.000
Negative	clean	5/50	FPR 0.100

The system has effectively two distinct competencies rather than one.

Panel detection is solved on this data: perfect recall on both panel-only and combined positives, with zero false positives on hard negatives and browser pages. The hard-negative construction succeeded — the model distinguishes an assistant chat panel from an Outline or Extensions panel in the same screen position without error, confirming it learned internal structure rather than position.

Ghost-text detection is materially weaker, at 0.700 recall. Every false positive in the entire evaluation occurred on clean editors — the class that differs from ghost-text positives by a single dimmed line. The discrimination the system finds hard is precisely the one predicted to be hard in Section 2.3.

8.7 Input Resolution Ablation

To test the hypothesis that ghost-text difficulty is driven by input resolution, an ablation was run changing only `img_size` from 512 to 768, with identical data, architecture, schedule and seed.

Ghost-vs-clean AUROC improves from 0.759 to 0.856. The recall and FPR columns must be read together to avoid a misleading conclusion: the 512-pixel model appears to have higher ghost recall, but achieves it while flagging 54% of clean editors. Its scores are simply shifted, so a fixed threshold of 0.50 sits at a different point on its curve. The threshold-free AUROC shows the 768-pixel model is better at every operating point — it attains comparable recall at a fifth of the false positive rate.

The mechanism is arithmetic. A ghost-text line rendered at 12–15 pixels in a 1680×1050 screenshot survives downscaling to 512 pixels as roughly four pixels of image height, before

Table 8.3: Resolution ablation on the held-out set. Only input resolution differs.

Metric (threshold 0.50)	512 px	768 px
Ghost-vs-clean AUROC	0.759	0.856
Ghost recall	0.825	0.700
Clean-editor FPR	0.540	0.100
Panel recall	1.000	1.000
Hard-negative FPR	0.000	0.000

the augmentations of Section 5.6 further blur and compress it. A chat panel occupying a third of the screen survives any downscaling comfortably. Higher input resolution preserves more of the only signal that distinguishes ghost text from a clean editor.

That 0.856 remains well short of the 1.000 achieved on panels indicates resolution is a contributing factor rather than the whole explanation, and motivates the tiling approach proposed in Section 10.3.

8.8 Operating Point Selection

Because the model ranks ghost text well but separates it imperfectly, the threshold choice materially changes system behaviour.

Table 8.4: Threshold sweep, 768-pixel model, held-out set. Panel recall is 1.000 and hard-negative and browser FPR are 0.000 at all three thresholds.

Threshold	Ghost recall	Clean-editor FPR
0.50	0.700	0.100
0.35	0.825	0.380
0.20 (shipped)	0.950	0.820

The shipped default is 0.20, chosen deliberately. The project objectives specify particular attention to minimising false negatives, and the review workflow makes the trade favourable: results are presented in descending score order and a reviewer dismisses a benign flag with a single keystroke, whereas a missed detection is invisible and unrecoverable. A reviewer who finds the queue too noisy can raise the threshold; they cannot recover a screenshot that was never surfaced.

What an 0.82 clean-editor false positive rate means in practice should be stated plainly rather than glossed. At this threshold the system is no longer *filtering* plain editor screenshots — it surfaces most of them. Its contribution for that class is *ranking*: genuine ghost text scores higher and sorts to the top of the queue. The structural negatives behave quite differently,

with browsers, quiz pages and non-chat panels correctly excluded at 0% false positives across all thresholds.

An accurate one-sentence characterisation of the delivered system is therefore: it eliminates browser pages, quiz pages and non-assistant panels entirely, detects assistant chat panels with perfect accuracy, and ranks plain editor screenshots by ghost-text likelihood for human review.

8.9 Threats to Validity

Synthetic evaluation. The quantitative results are measured on synthetic data. Although the held-out set uses an unseen seed and unseen sessions, it shares a generator with the training data — the same drawing code, the same four themes, the same fonts. Real screenshots differ in ways the generator does not model: unusual themes, extensions altering the interface, partial renders, non-standard window sizes. The reported figures should be read as an upper bound on real-world performance.

Small real sample. The OCR baseline rests on 19 real positives. The direction of that result is unambiguous — a detector that structurally cannot see ghost text will not see it — but the precise recall figure of 0.214 has wide error bars.

No real-data CNN evaluation. The CNN has not been evaluated quantitatively on real screenshots, because no labelled real set of sufficient size exists. This is the most significant limitation of the evaluation and follows directly from the data access problem in Section 5.2.

Single architecture. Only ResNet50 was evaluated. No comparison against other backbones or a vision transformer was performed.

Chapter 9

Development Experience

9.1 Chapter Overview

This chapter reflects on the process of building the system rather than its final state. Section 9.2 describes approaches that proved valuable. Section 9.3 documents the significant problems encountered and how they were diagnosed, and Section 9.4 states what I would do differently.

9.2 What Worked

Narrowing the scope. The most valuable decision was abandoning the general “suspicious screenshot” framing for two specific tasks. It converted a project I could not evaluate into one I could measure, and every subsequent result in this report depends on it. I initially resisted the change because the narrower version felt less ambitious; in retrospect the general version was not more ambitious, merely less well defined.

Measuring instead of assuming. Several plausible optimisations turned out to be wrong, and only measurement revealed it. Parallelising OCR made it six times slower. Downscaling before OCR was nearly twice as fast and silently broke detections. Cropping to a region of interest was faster and lost true positives. In each case my reasoning was sound and the outcome was the opposite. I now treat “this should be faster” as a hypothesis rather than a conclusion.

Building diagnostics rather than reading aggregate numbers. The per-variant report was the single most useful tool I built. Before it existed I had a model with 0.94 AUROC and no idea whether it detected ghost text at all. Afterwards I could see that panel detection was perfect and ghost detection was at 0.70, which reframed the entire remaining effort. Aggregate metrics told me the model was good; the breakdown told me what it was good at.

Regression tests for reasoning, not just code. Several tests exist to preserve decisions

whose rationale is invisible in the code — that a low CNN score must never suppress OCR, that a filename must not trigger a keyword. Each encodes a subtle argument that a future reader, including me, could easily undo.

9.3 Problems Encountered

9.3.1 A blank application window

The first run of the interface produced a window in which the header and title simply did not render, showing content bleeding through from behind. The layout code was correct and the same code behaved correctly elsewhere.

The cause was the Python interpreter. macOS ships an Xcode Python 3.9 bundled with Tk 8.5.9, a version with known rendering defects on modern macOS; widgets constructed alongside a `PhotoImage` load intermittently fail to paint. Rebuilding the virtual environment against Homebrew Python 3.11 with Tk 8.6 resolved it completely.

The lesson was about diagnosis rather than Tk. I spent time inspecting my layout code because that was what I had written and therefore what I assumed was wrong. The evidence — content from another window showing through, which no layout bug can cause — pointed at the rendering layer from the beginning. I have since recorded the interpreter requirement in the project README so the problem cannot recur silently for anyone else.

9.3.2 A repository that could not be pushed

Several weeks in, pushing to GitHub failed: a file exceeded the 100 MB limit. The file was `libtorch_cpu.dylib`, part of PyTorch, inside a virtual environment that had been committed before a `.gitignore` existed.

Because the offending commits had never been pushed, the fix was to reset to the last shared commit and recreate the work as a single clean commit respecting `.gitignore`. Adding the directory to `.gitignore` alone would not have helped, since the file remained in history.

The underlying mistake was committing before thinking about what belonged under version control. I subsequently accumulated four virtual environments totalling 2.4 GB, three of them created while diagnosing the Tk problem and never removed.

9.3.3 A detector that detected itself

Testing the keyword detector on real screenshots produced false positives on images containing no assistant. The cause was that my own source file is named `copilot.py`, it was visible in the editor’s file explorer, and the bare keyword `copilot` matched it.

This is my favourite bug in the project because the detector was working perfectly — it found the string “copilot” on screen, which is exactly what it was told to do. The specification was

wrong, not the implementation. It clarified something about keyword detection generally: the detector has no concept of *where* text appeared or what interface element it belonged to, and any such system will match text in file listings, bookmarks and documents with equal enthusiasm.

9.3.4 Correct rules that were globally wrong

Two bugs shared a structure worth naming. The Brightspace detector flagged every VS Code screenshot in combined mode, because a screenshot with no quiz markers is a tab-leave — correct in a browser-only examination, wrong when the student is legitimately in an editor. Separately, the off-task keyword list flagged legitimate quiz pages whose questions mentioned Wikipedia or Google.

In both cases the rule was locally correct and the surrounding assumption was unstated. This is the class of bug I found hardest to anticipate, because testing a component in isolation cannot reveal it. Both were found by running the full application on realistic input rather than by unit testing.

9.3.5 Evaluating on training data

The first version of the per-variant report scored every image in the dataset, including the 560 the model had trained on. The numbers it produced were inflated and, worse, plausible — nothing about the output indicated anything was wrong.

The fix was to have the tool reproduce the training split exactly and report only held-out sessions by default. That an evaluation tool can silently produce meaningless-but-believable numbers is a lesson I expect to carry well beyond this project.

9.3.6 Data access

The problem with the largest effect on the final result is the one I made least progress on. The proposal identified real CoMas data as the primary source and correctly named it as the project’s principal risk. I did not begin the access process early enough, and the timeline for institutional data agreements and ethics review is not something a student can compress.

The synthetic fallback was anticipated in the proposal and does work, but every number in Chapter 8 carries an asterisk as a result. If I were starting again, the access request would be the first task of week one, before any code was written, precisely because its lead time is long and entirely outside my control. Risks that cannot be compressed later must be started first.

9.4 What I Would Do Differently

Start the data request immediately. As above — this is the change that would most improve the work.

Build the per-variant evaluation before the model. I trained for several runs while only able to see a single aggregate number. Had the breakdown existed from the start, I would have identified the ghost-text weakness far earlier and had more time to address it.

Question resolution earlier. The input size of 512 pixels was carried forward without examination for most of the project. It is the single hyperparameter most directly implicated in the model’s main weakness, and one afternoon’s ablation produced the clearest result in the report. I should have asked what the signal physically looks like after preprocessing much sooner.

Set up version control properly on day one. A `.gitignore` written before the first commit would have avoided an afternoon of history surgery and 2.4 GB of accumulated virtual environments.

Chapter 10

Conclusion

10.1 Summary

This project delivered a working screenshot triage system for CoMas comprising a hybrid detection pipeline, two synthetic data generators, a reviewer application and an evaluation toolchain, supported by 42 automated tests.

Assessed against the objectives of Section 1.3:

Objective 1 (dataset) — partially met. A labelled dataset was assembled, but synthetically rather than from real CoMas screenshots. The proposal’s stated fallback became the primary strategy.

Objective 2 (classifier) — met. A ResNet50 classifier was designed, trained with transfer learning, and integrated into the application.

Objective 3 (evaluation) — met on synthetic data, not met on real data. Precision, recall, F1 and AUROC are reported, along with a per-variant breakdown and a resolution ablation. No quantitative evaluation on real screenshots was possible.

Objective 4 (inference pipeline) — met and exceeded. The delivered application provides ranked review, adjustable sensitivity, triage state and evidence export.

The most substantive technical finding is that the two visual manifestations of an AI coding assistant require different detection methods, and that the harder of the two is resolution-limited in a way that can be measured and quantified rather than merely asserted.

10.2 Limitations

Synthetic evaluation only. The principal limitation, discussed in Section 8.9.

Ghost-text ceiling. At 0.856 ghost-vs-clean AUROC the model ranks ghost text well but does not separate it cleanly. The shipped operating point recovers 95% of ghost-text screenshots at the cost of surfacing most clean editors for review.

Keyword maintenance. Interface strings change with vendor updates. The keyword list should be reviewed each term, and a strong argument for retaining the CNN is that it does not share this failure mode.

Two tools only. Only Copilot and Cursor are covered. Other assistants would be missed by the keyword detector, though the CNN may generalise to visually similar panels.

Single-screenshot analysis. Each screenshot is judged independently. Session-level patterns — a student whose captures show an assistant repeatedly across an examination — are not modelled.

10.3 Future Work

Obtain real data. Everything else is secondary. A labelled corpus of real CoMas screenshots would allow genuine evaluation and would very likely improve the model, since real screenshots contain variation no generator reproduces.

Tiled inference for ghost text. Rather than downscaling a whole screenshot to 768 pixels, divide it into overlapping tiles at native resolution, score each, and take the maximum. Ghost text would then be evaluated at full resolution rather than compressed by a factor of four. The resolution ablation gives direct evidence that this is the most promising avenue.

Session-level analysis. Filenames already encode session identifiers. Aggregating scores per student and presenting a timeline would match how proctors actually think — about students, not individual images — and would allow weak per-image evidence to accumulate into a strong per-session signal.

Localisation. Highlighting the region responsible for a detection, via attention maps or the tiling above, would let a reviewer verify a flag at a glance rather than searching the screenshot for what triggered it.

Persisting triage state. Triage decisions are currently held in memory and lost when the application closes. Persisting them would support review sessions spanning multiple sittings.

Bibliography

- [1] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [2] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A Large-Scale Hierarchical Image Database,” *IEEE Conference on Computer Vision and Pattern Recognition*, 2009.
- [3] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *Advances in Neural Information Processing Systems 32*, 2019.
- [4] R. Smith, “An Overview of the Tesseract OCR Engine,” *Ninth International Conference on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.
- [5] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” *International Conference on Learning Representations (ICLR)*, 2019.
- [6] T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” *PLoS ONE*, vol. 10, no. 3, 2015.
- [7] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, “How Transferable Are Features in Deep Neural Networks?” *Advances in Neural Information Processing Systems 27*, 2014.

Appendix A

Repository Structure

```
1 Honours Project/
2   comas/                                main package
3     __init__.py
4     brightspace.py                      Brightspace detector
5     config.py                           typed YAML configuration
6     copilot.py                           VS Code detector, CNN scorer
7     data.py                             dataset, transforms, splitting
8     evaluate.py                          metric computation
9     gui.py                              desktop reviewer interface
10    infer.py                             command-line inference
11    model.py                             ResNet50 and fusion models
12    ocr.py                               Tesseract wrapper and cache
13    realpairs.py                         real screenshot utilities
14    synthetic.py                         VS Code screenshot generator
15    synthetic_brightspace.py            browser screenshot generator
16    train.py                             training loop
17    variant_report.py                    per-variant evaluation
18    assets/carleton_logo.png
19  tests/                                42 automated tests
20  data_vscode/                          generated training data
21  data_vscode_holdout/                  generated evaluation data
22  data_brightspace/                     generated browser data
23  checkpoints/                          trained model weights
24  cache/ocr.json                        OCR cache
25  config.yaml                           configuration
26  requirements.txt
27  README.md
```

Listing A.1: Project layout. Generated data and checkpoints are excluded from version control.

Appendix B

Configuration Reference

```
1 data:
2   root: data_vscode/
3   img_size: 768
4   num_workers: 4
5   batch_size: 8
6   val_frac: 0.15
7   test_frac: 0.15
8   seed: 42
9
10 model:
11   use_ocr: false
12   cnn_backbone: resnet50
13   text_encoder: sentence-transformers/all-MiniLM-L6-v2
14   fusion_hidden: 256
15   dropout: 0.3
16
17 train:
18   epochs: 10
19   lr: 1.0e-4
20   weight_decay: 1.0e-4
21   pos_weight_auto: true
22   pos_weight_override: null
23
24 paths:
25   checkpoint: checkpoints/vscode_768.pt
26   ocr_cache: cache/ocr.json
27
28 logging:
29   level: INFO
```

Listing B.1: config.yaml as used for the final 768-pixel model.

Appendix C

Instructions to Run and Test

C.1 Environment

Python 3.11 is required. On macOS the system Python bundled with Xcode must *not* be used, as its Tk 8.5 has rendering defects that prevent the interface from displaying correctly (Section 9.3.1).

```
1 brew install python@3.11 python-tk@3.11 tesseract
2 python3.11 -m venv .venv
3 source .venv/bin/activate
4 pip install -r requirements.txt
5 python3 -c "import tkinter; print(tkinter.TkVersion)" # must print 8.6
```

C.2 Running the Application

```
1 python3 -m comas.gui
```

Select a detection mode, upload screenshots, and review the ranked results. Keyboard shortcuts in the review screen: arrow keys navigate, **r** marks reviewed, **d** dismisses, **c** clears, **e** exports.

C.3 Regenerating Data and Retraining

```
1 # generate datasets
2 python3 -m comas.synthetic --out data_vscode --n-active 200 --n-clean 200
3 python3 -m comas.synthetic_brightspace --out data_brightspace
4 cp data_brightspace/on_quiz/*.png data_brightspace/left_quiz/*.png \
5   data_vscode/no_copilot/
6
```

```
7 # train
8 python3 -m comas.train
9
10 # held-out evaluation
11 python3 -m comas.synthetic --out data_vscode_holdout \
12     --n-active 100 --n-clean 100 --sessions 10 --seed 777
13 python3 -m comas.variant_report --data data_vscode_holdout \
14     --split all --threshold 0.20
```

C.4 Running the Tests

```
1 python3 -m pytest tests/ -q
```

Expected result: 42 passed. Tests requiring Tkinter are skipped automatically in environments without a display.

C.5 Command-Line Detection

```
1 python3 -m comas.copilot --folder path/to/screenshots --out ranked.csv
2 python3 -m comas.brightspace --folder path/to/screenshots --out bs.csv
```